

- Given a Boolean function f , find an assignment of input variables that makes f TRUE.
- First NP-complete problem
- Basis of synthesis, formal verification, ATPG, routing in FPGAs

Conjunctive Normal Form (CNF)

- Literal: a Boolean variable in its *true* or *negated* form
- Clause: disjunction of one or more literals (OR, $\vee$)
- CNF: Conjunction of clauses (AND, $\wedge$)
- AKA Product of Sums
- A popular format has the namesake extension (`.cnf`):
e.g.
$$f = (x_1 \vee \neg x_2 \vee x_3) \wedge (x_2 \vee \neg x_3 \vee x_4)$$
- Solution/Model:
{TRUE, TRUE, TRUE, TRUE} expressed
as:
`-1 -2 -3 -4 0`

```
c "c": line comment
c A sample CNF file
c p: problem specified as "p cnf N M"
c N: num variables
c M: num clauses
c 0: reserved clause terminator
c literal: i(true form), -i(negated form)
c 1 <= i <= N
c clause: all literals till terminator
c % 0 for problem termination
p cnf 4 2
1 -2 3 0
2 -3 4 0
%
0
```

Tseytin encoding

- Consider $f = \bigwedge_{i=1}^p c_i$ and $g = \bigwedge_{j=1}^q d_j$
- $h = f \vee g$
- Brute for distributive law application $\implies$ explosion in #clauses ($p \cdot q$):

$$h = \bigwedge_{i=1}^p \bigwedge_{j=1}^q (c_i \vee d_j)$$

- Instead introduce variables x and y :

$$\begin{aligned} h &= (\neg x \vee \neg y) \wedge (\neg x \vee f) \wedge (\neg y \vee g) \\ &= (\neg x \vee \neg y) \bigwedge_{i=1}^p (\neg x \vee c_i) \bigwedge_{j=1}^q (\neg y \vee d_j) \end{aligned}$$

- Linear increase in #clauses

Davis-Putnam-Logemann-Loveland(DPLL) algorithm

- Pure literal: literal that appears in pure form in the entire CNF(all true or all negated)

```
function DPLL( $f, m$ )  $\triangleright f = \text{CNF}; m = \text{partial assignment}$   
  while  $l \in \text{UnitClause}(f, m, \text{negated})$  do  $\triangleright$  list of true / negated unit literals  
     $m \leftarrow m \cup l$   
     $f \leftarrow f \setminus l$   $\triangleright$  remove  $\neg l$  from clauses and clauses with  $l$   
  while  $p \in \text{PureLiterals}(f)$  do  
     $m \leftarrow m \cup p$   
     $f \leftarrow f \setminus p$   
  if  $\text{FALSE} \in f$  then  
    return FALSE  
  if  $f = \emptyset$  then  
    return TRUE  
   $l \leftarrow \text{PickBranchingLiteral}(f, m)$   
  return DPLL( $f, m \cup l$ ) or DPLL( $f, m \cup \neg l$ )
```